

Family 10: the granularity-aware tensor, as built one registry, three storage tiers, and four observation stores 2.12 billion measurements — what shipped, measured

blauewelt/earth — design note

15 September 2026

Abstract

Family 10 is the programme’s storage contract for data of mixed granularity. Nothing is re-sampled onto a common grid when it is stored: every source keeps its own resolution and cadence, and the token a model reads carries the measurement’s *footprint* — how much area and how much time it averaged — beside its offset from the place and pentad being asked about. One registry file, `tensors/family10/family10.json` on the public Hugging Face dataset repository `chfrank/earth-tensors`, lists every group of the family and says which of three storage tiers it belongs to: **G**, gridded dense fields held at their native grid and read by a byte range (the four channel groups of family 7.2, referenced rather than copied); **P**, points, profiles and tracks held as columns sorted by five-day bin with a compressed-sparse-row index and read by a k -nearest search; and **T**, tiles, which is designed and not built. Experiment E-079 built the four tier-P stores: **gdp** (the Global Drifter Program’s six-hourly surface drifters, 48,480,798 rows), **gtmba** (the tropical moored arrays TAO/TRITON, PIRATA and RAMA, one row per mooring per day, 1,001,282 rows), **socat** (the Surface Ocean CO₂ Atlas, 41,830,675 underway ship and mooring measurements) and **slatrack** (Copernicus Marine’s reprocessed along-track sea-level anomaly, 2,030,800,150 samples from 29 altimeter mission datasets). All four were published and then independently verified from the bytes on 2026-09-14, for \$0.55. This note states what the registry says, what a store is, what each store contains, how the builds ran, how to read one, and what the layout still cannot promise. A final section is a self-contained handover for an external agent that reads the public dataset and never writes to it.

1 Purpose: four prediction goals, served by measurements

The programme tracks four prediction goals: the El Niño–Southern Oscillation (the tropical Pacific’s warm-and-cold swing, read out as the Niño-3.4 sea-surface temperature index), the ocean carbon sink (how much carbon dioxide the sea surface absorbs), the Atlantic Meridional Overturning Circulation or AMOC (the northward heat-carrying flow of the Atlantic, read out as transport at 26.5° N), and sea-surface temperature itself. Everything the programme had fed a model before family 10 was a *gridded product* — a field someone else had already interpolated onto a regular grid. Family 10’s four tier-P stores are the measurements themselves.

The design argument is in the family-10 design note (E-078) and is three measurements rather than three preferences. A coarse value copied into fine cells is one measurement presented as many, and a model cannot tell nine readings that agree from one reading copied nine times. A fine value averaged into a coarse cell discards exactly what the finer source was bought for. And distance is a feature that a grid hides: from the same five nearest Argo profiles, optimal interpolation beat climatology by 21 % on temperature where the gridded monthly product, read at the nearest cell, gave a learned codec 3 % (E-076b).

2 Three tiers, one registry

Sources are stored by *density*, because density decides what a read costs — not by their sphere or their science.

store	what it measures	the goal it serves, and why this rather than a grid
gdp	surface buoy velocity at 15 m and sea-surface temperature, every six hours	AMOC : observed velocity — the quantity a learned codec could not recover from gridded fields (E-069). Also an independent temperature instrument
gtmba	one row per tropical mooring per day: surface and air temperature, salinity, wind, current, and subsurface temperature at eleven depths to 500 m	El Niño : the warm-water volume and thermocline depth, where the six-month lead lives. Warm-water volume is computed by the consumer from the 20 °C isotherm depth at the equatorial sites, not stored: it is a derived quantity of a choice of sites
socat	surface-water carbon-dioxide fugacity from ships and moorings, with temperature, salinity and barometric pressure	carbon : the only direct observation of the quantity — every global sink estimate is a reconstruction fitted to this archive
slatrack	sea-level anomaly along each altimeter's ground track, one sample per second (≈ 7 km)	AMOC as geostrophic transport measured rather than modelled, and El Niño as Kelvin waves in sea level. Also fronts, for sea-surface temperature

Table 1: The four tier-P stores against the four prediction goals.

tier	what	layout	state
G — gridded dense	family 7.2's four channel groups: ocean surface, atmosphere and land, ocean interior, ocean colour	one bin-major <code>.npy</code> per group at its <i>native</i> grid; a read is one HTTP byte range of one bin	built; referenced by the registry, never copied
P — points, profiles, tracks	Argo profiles (family 8) plus the four stores of this note	columns sorted by (<code>bin</code> , <code>time_days</code>) with compressed-sparse-row bin offsets; a read is a <i>k</i> -nearest search	built: five groups
T — tiles	fine imagery and swaths (Sentinel-1/2, Landsat, NISAR)	not values but a tile-date catalogue plus a small per-tile codec's cached tokens, held <i>as</i> a tier-P store	designed, not built

Table 2: The three storage tiers, and what exists.

2.1 The registry, and its exact shape

`tensors/family10/family10.json` on `chfrank/earth-tensors` is the single entry point. The copy this note describes was regenerated **2026-09-14T22:04:04Z** by the `slatrack` build's last step from builder commit `6d67855`; it is 45,610 bytes and carries `n_groups: 9` with `groups_missing: []` — the first time the family has been complete. Its top level:

```
{ "family": "family10", "repo": "chfrank/earth-tensors",
  "epoch": "1982-01-01", "pentad_days": 5,
  "bin_rule": "bin = floor((date - 1982-01-01) / 5 days); a tier-P store may
               hold NEGATIVE bins ... and its CSR index runs over its own
               bin_first .. bin_first + n_bins - 1",
  "tiers": { "G": ..., "P": ..., "T": "... NOT BUILT" },
  "token_schema": { "fields": ["value[C]", "mask[C]", "dx_km", "dy_km",
                              "dt_days", "log2_fp", "log2_dt", "n_R", "source",
                              "channel"],
                  "log2_fp": "log2(footprint_km / 27.83)",
                  "log2_dt": "log2(support_days / 5)", "clamp": [-12, 12] },
  "readers": { "G": "ml/cone_sampler.py", "P": "ml/family10_store.py" },
  "tier_g": { "stem": "family7_global025_pentad_12", "recipe": "f712",
              "manifest": ".../manifest.json", "manifest_present": true,
              "fallback_note": null, "built_at": ..., "sources": {...} },
  "groups": [ ... 9 entries ... ], "groups_missing": [], "n_groups": 9,
  "builder_git_sha": "6d67855...", "generated_utc": "2026-09-14T22:04:04Z",
  "design": "...E078...", "plan": "...E079...", "handover": "...HANDOVER.md" }
```

A consumer dispatches on tier and needs nothing else. Every group entry carries `name`, `tier`, `cadence`, `C`, `channels` (name, unit, label), `footprint`, `layout`, `prefix`, `repo` and its files with their

sha256. A tier-G entry adds the grid geometry, **shape**, **dtype**, the **.numpy** header length and one bin’s slab size; a tier-P entry adds **N**, its own bin range and live-bin count, the date range and per-year counts, the quality-control policy and its cut-off, and the platform-identifier rule. The two **layout** strings are the dispatch in one sentence each: “one bin-major **.numpy** at the group’s **NATIVE** grid; a read is one **HTTP** range” against “columns sorted by (**bin**, **time_days**) with **CSR** bin offsets; a read is a *k*-nearest search, one-sided in time”.

group	tier	cadence	<i>C</i>	bin_first	<i>N</i> or shape	files
g025	G	pentad	7	0	$3142 \times 721 \times 1440$	1
g100	G	pentad	15	0	$3142 \times 181 \times 360$	1
oc025	G	pentad	2	1145	$1997 \times 721 \times 1440$	1
rg100	G	monthly	32	0	$252 \times 181 \times 360$ (live bins only)	1
argo	P	irregular	32	0	2,678,439	12
gdp	P	6-hourly	4	−211	48,480,798	9
gtmba	P	daily	18	−304	1,001,282	9
socat	P	irregular	4	−1768	41,830,675	9
slatrack	P	irregular	3	803	2,030,800,150	9

Table 3: The nine groups of the registry as regenerated 2026-09-14T22:04:04Z. The five tier-P stores hold 2,124,791,344 observations; the four of E-079 hold 2,122,112,905 of them.

Tier G is referenced, not copied. The **tier_g** block names *which* family-7 manifest it describes — stem **family7_global025_pentad_12**, recipe **f712** — and the four group entries carry that manifest’s own file hashes and byte counts, compared against it digest by digest in the independent verification: **4 of 4 match**, **oc025** (8,293,461,248 bytes) included. Two properties of the registry are load-bearing and both were exercised before the world caught up with them. A group the builder cannot read is *not in groups* and *is in groups_missing* — **slatrack** was there for most of 2026-09-14 — because a registry that listed a group it could not describe would be worse than a short one. And when family 7.2’s manifest was still a 404, the morning’s registry described the earlier **f710** tensor and said so in **tier_g.fallback_note** rather than silently describing a different tensor. Both mechanisms are retired by the tensor arriving: **fallback_note** is **null**.

3 The token, and the two footprint fields

Every observation, from every tier, becomes one token:

$$\text{token} = (\mathbf{v}[C], \mathbf{m}[C], \Delta x, \Delta y, \Delta t, \log_2 f_p, \log_2 f_t, n_R, s, c) \quad (1)$$

where **v** is the measurement with mask **m** marking which of its *C* components were measured; ($\Delta x, \Delta y$) the offset from the anchor in kilometres east and north and Δt the age in days, measured back from the *end* of the anchor’s bin; n_R the number of observations inside the search bounds; and *s, c* learned embeddings of the source class and the channel. The two footprint fields are

$$\log_2 f_p = \log_2 \frac{\text{footprint in km}}{27.83 \text{ km}}, \quad \log_2 f_t = \log_2 \frac{\text{support in days}}{5 \text{ days}}, \quad (2)$$

the spatial and temporal support in units of one 0.25° cell (27.83 km, being 0.25° of latitude at 111.32 km per degree) and one pentad, clamped to ± 12 . They carry what the offsets cannot: a value 30 km from a 1.9° reanalysis average and one 30 km from a point measurement say different things about the anchor, because the coarse value is a smoother, lower-variance quantity.

The $\log_2 f_p = -4$ floor is a clamp, not a measurement: a drifter, a mooring, a ship intake and an Argo float are all points at any scale this grid resolves. **slatrack** is the one tier-P store whose spatial footprint is a real number — $-2.0 = \log_2(7/27.83)$ — and the verification measured its along-track ground spacing at **6.2–6.5 km**, so the column is not a number someone typed.

4 What a tier-P store is

A store is one directory per source under **tensors/family10/**, holding nine arrays in **.numpy** format and one **store.json**. Every store has exactly these files, with exactly these dtypes, whatever the source.

source	$\log_2 f_p$	$\log_2 f_t$	why
gdp — a drifter fix	-4.0	-4.3	a point; a six-hour sample, $\log_2(0.25/5)$
gtmba — a mooring-day	-4.0	-2.3	a point; a daily mean, $\log_2(1/5)$
socat — an underway sample	-4.0	-4.0	a point; minutes, at the clamp floor
slatrack — one 1 Hz sample	-2.0	-4.0	$\log_2(7/27.83)$; one second, at the floor
argo — a profile	-4.0	-4.0	a point taken in minutes
g025 — a 0.25° pentad cell	0.0	0.0	the unit both fields are defined against
g100 — a 1° pentad cell	+2.0	0.0	the reanalysis, stored at the grid it is on
rg100 — a 1° monthly field	+2.0	+2.6	$\log_2(30.4/5)$

Table 4: The footprint fields of every group in the registry. They are constant per group for the dense tiers and constant per store for the four built tier-P stores — but stored per row, so a store that later mixes supports reads identically.

file	dtype, shape	what
bin.npy	int16 $[N]$	the five-day bin, <code>floor((date - 1982-01-01)/5 d)</code> . Signed, and negative before 1982
time_days.npy	float32 $[N]$	days since 1982-01-01T00:00 UTC, fractional, negative before it
lat.npy	float32 $[N]$	degrees north
lon.npy	float32 $[N]$	degrees east, in $[-180, 180)$
values.npy	float16 $[N, C]$	the channels, in raw units ; NaN means not measured
platform.npy	int64 $[N]$	the platform: a numeric source id where the archive has one, else <code>int(sha1(id)[:15], 16)</code> — 60 bits, never negative, deterministic across machines and Python versions (<code>hash()</code> is not)
qc.npy	uint8 $[N]$	0 not assessed, 1 good, 2 probably good, 3+ the source’s worse grades
fp.npy	float16 $[N, 2]$	$(\log_2 f_p, \log_2 f_t)$, per row
bin_offsets.npy	int64 $[B + 1]$	compressed-sparse-row offsets over <i>this store’s own</i> bin range
store.json	JSON	the schema, footprint, quality-control policy, provenance, per-year counts, per-channel statistics, and the sha256 of every file above

Table 5: The nine arrays. One row costs $27 + 2C$ bytes plus the `.npy` headers: 33 for **slatrack** ($C = 3$), 35 for **gdp** and **socat**, 63 for **gtmba**.

The bin, and why it may be negative. A bin is a five-day period counted from 1982-01-01, so `bin = [(date - 1982-01-01)/5 d]`. Drifters begin in 1979 and the carbon archive in 1957, and those rows are **kept, with negative bins**, so the store is the whole record and the *loader* clips to whatever axis a tensor has — truncating at build time cannot be undone. The reader’s definition is a true floor on floats, which is the whole reason the negative side works: `np.floor(days / 5)` puts -0.5 days in bin -1 , where integer division would put it in bin 0.

The index. `bin_offsets` is a compressed-sparse-row index: the rows of bin b are `off[b - bin_first] : off[b - bin_first + 1]`, with `off[0] == 0` and `off[-1] == N`. **bin_first is not zero** — it is -1768 for **socat** and $+803$ for **slatrack** — and code that indexes by `bin` without subtracting it will either crash or, worse, wrap.

The sort order is part of the contract. Rows are sorted by `(bin, time_days)` ascending. The bin order is what makes the index valid at all; the within-bin time order is what lets a reader stop at the anchor’s own bin and still have the most recent observations at the end of the slice.

The longitude invariant, and the float32-cast lesson. `lon` is in $[-180, 180)$, half-open, and the dateline is not a wall: the reader wraps the longitude difference, and a consumer computing offsets by hand must too. Enforcing the half-open interval cost two fixes on the same day, and they are one lesson. The first (**61ab4fa**) folds a source sample at exactly $+180.0^\circ$ E back into the interval; the second (**fb5d5ab**) moves the wrap to *after* the float32 cast, because a source longitude of 179.99999 wraps to itself in float64 and **rounds up to** 180.0 when it is stored in float32 — which the build’s own assertion then, rightly, refused. **An invariant has to hold in the column’s dtype, not in the wider type it was computed in.** Both fixes are visible in the published bytes: over **slatrack**’s 2.03 billion rows the maximum longitude is 179.99998474 — `nextafter(180, 0)` in float32 — the minimum is exactly -180.0 , and not one row sits at $+180.0$.

The float32 timestamp. `time.days` is float32 and runs to $\approx 15,700$ days, where its spacing is $2^{-10} \text{d} = 84.4$ seconds. The builder therefore computes `bin` from the float32 value that is *actually stored*, not from the float64 the parser had: otherwise a row would sit in bin b carrying a timestamp that reads as bin $b + 1$, and the search — which requires $\Delta t = 5(b + 1) - t \geq 0$ — would silently never return it for its own anchor. The identity `bin == floor(time.days / 5)` holds on every row of all four stores. Section 6.4 says what the 84 seconds cost.

Raw units, on purpose. Nothing is z-scored and nothing is anomalised: a measurement already divided by a training-set standard deviation is no longer one. Standardise in the loader, from training bins only — a statistic computed over the whole store carries the test years into the weights, and that leak is invisible.

5 The four stores

All four were built on 2026-09-14 and then **independently verified the same day by a session that did not build them**: every file fetched from the Hub and hashed against `store.json`’s own record, then every structural property and every statistic *recomputed from the arrays* rather than read out of the metadata claiming them (`docs/FAMILY10-VERIFICATION-2026-09-14.md`). Physical bounds are shared: $|u|, |v| < 5 \text{ m s}^{-1}$; $-3 < \text{SST} < 45^\circ\text{C}$; $0 < f\text{CO}_2 < 2000 \mu\text{atm}$; $|\text{sla}| < 3 \text{ m}$; $0 \leq \text{sss} \leq 45 \text{ PSU}$; $|\text{wind}| < 120 \text{ m s}^{-1}$; $-60 < \text{airt} < 60^\circ\text{C}$. A value outside is set to **NaN and counted**, never clipped — clipping puts a fabricated number where a broken instrument was.

5.1 gdp — the Global Drifter Program

source	NOAA’s Atlantic Oceanographic and Meteorological Laboratory, ERDDAP table drifter_6hour_qc — “Global Drifter Program, 6 Hour Interpolated QC Drifter Data”
cadence	six-hourly: positions and velocities kriged onto 00/06/12/18 UTC. Interpolated, not raw fixes — the footprint $(-4.0, -4.3)$ says “a point, a quarter-day sample”, not “an instantaneous measurement”
channels ($C = 4$)	<code>u</code> , <code>v</code> (m s^{-1} , eastward and northward at the 15 m drogue depth); <code>sst</code> ($^\circ\text{C}$, the buoy’s own thermistor); <code>drogue</code> (1 where the fix precedes the drifter’s <code>drogue_lost_date</code> , 0 after it, NaN where that date is 1970-01-01, which the archive documents as “uncertain from the beginning”)
quality control	no per-row flag is published, only per-sample error estimates, so the flag is <i>derived</i> here rather than in a consumer: position error = $\text{hypot}(\text{err}_{\text{lat}}, \text{err}_{\text{lon}} \cos \phi) \times 111.32 \text{ km}$; over 50 km the row is dropped and counted, $\leq 5 \text{ km}$ is <code>qc 1</code> , the rest <code>qc 2</code>
rows	$N = 48,480,798$ of 48,708,540 read (99.53%): <code>drop_pos_err</code> 227,174, <code>drop_no_values</code> 568, out-of-bounds <code>sst</code> 5,028 and <code>v</code> 1
bins	$-211 \dots 3141$, 3,353 of 3,353 live (100 %) ; 1979-02-15T00:00Z $\rightarrow$ 2024-12-31T18:00Z
years	46, no gaps: 1979 = 24,046 rows, 2020 = 2,301,964, 2024 = 1,834,400, with a twenty-fold rise between 1986 (47,898) and 1993 (767,697)
platforms measured	28,689 distinct drifters, by their AOML identifier 0.963865 overall; <code>u</code> 0.998862, <code>v</code> 0.998861, <code>sst</code> 0.886963, <code>drogue</code> 0.970775. <code>qc</code> is 1 on 93.571 % of rows and 2 on 6.429 %
store.json	built 2026-09-14T09:46:18Z at commit 0d5860d; sha256 c3a14beb8d51db73f33636c9b805509e9d71721c9fe92d876b8c9621d4ab1e10
verdict	PASS on every check : 9 of 9 hashes, every structural property, all four channels reproduced, and the per-year counts reproduced exactly from <code>time.days</code> for 46 of 46 years

`drogue` is a channel and not a filter, deliberately: a drogued drifter measures the 15 m current and an undrogued one the surface, and an *unknown* drogue is a third state, NaN rather than 0. The verification confirmed exactly three states and no fourth value (1 on 21,476,927 rows, 0 on 25,587,043, NaN on 1,416,828), and a global mean drift of -0.00013 and $+0.0032 \text{ m s}^{-1}$ — what a global drifter ensemble should give.

5.2 gtmba — the tropical moored arrays

The smallest store of the four and the widest, and the only one whose rows are a fixed lattice rather than a moving platform.

Two physical cross-checks were run rather than assumed, and both passed. `t_1m` and `sst` come from two *different* PMEL datasets joined on (`station`, `day`), and their store-wide means agree to 0.0003°C .

source	NOAA’s Pacific Marine Environmental Laboratory: TAO/TRITON in the Pacific, PIRATA in the Atlantic, RAMA in the Indian Ocean, through the Observing System Monitoring Center ERDDAP (which answers a 302 to the CoastWatch server). Nine <code>pmelTaody*</code> datasets are listed; the six publishing the stored quantities are read and joined on (<code>station</code> , <code>day</code>)
cadence	daily: one row is one mooring on one day , which is what the array is designed to deliver. Footprint $(-4.0, -2.3)$
channels ($C = 18$)	<code>sst</code> , <code>sss</code> , <code>airt</code> , <code>wind_u</code> , <code>wind_v</code> , <code>u_cur</code> , <code>v_cur</code> , then subsurface temperature at eleven standard depths <code>t_1m</code> , <code>t_20m</code> , <code>t_40m</code> , <code>t_60m</code> , <code>t_80m</code> , <code>t_100m</code> , <code>t_120m</code> , <code>t_140m</code> , <code>t_180m</code> , <code>t_300m</code> , <code>t_500m</code> . Units $^{\circ}\text{C}$, PSU, $^{\circ}\text{C}$, $\text{m s}^{-1} \times 4$, then $^{\circ}\text{C}$. A site that does not carry a depth leaves that channel NaN; nothing is interpolated in
quality control	a code beside every daily value (0 missing, 1 highest, 2 default, 3 adjusted, 4 lower, 5 sensor failed). A <i>sample</i> is kept only at 1 or 2; a <i>row</i> survives if any channel does, and its <code>qc</code> is the worst code among the survivors. Wind components share the wind-speed code, the only one published for them; depths match the eleven levels within 0.6 m
rows	35,771,817 samples read $\rightarrow$ 15,781,023 kept, joined into $N = 1,001,282$ mooring-days. <code>drop_fill</code> 15,835,928 (the 10^{35} fill), <code>drop_depth</code> 3,261,857, <code>drop_qc</code> 893,009; 37 dataset-year requests came back empty
bins	$-304 \dots 3141$, 3,386 of 3,446 live (98.3%); 1977-01-01 $\rightarrow$ 2024-12-31
years	48: 1977 = 173 rows, 1992 = 16,277 as TRITON and PIRATA come in, a plateau of 33,000–38,000 in 2007–2012, then a decline to 2024 = 20,958
platforms measured	159 distinct moorings, by World Meteorological Organization code; latitudes span $[-25, 21]$ only 0.568374 overall — the lowest of the four, and honestly so: the current channels exist on 6.67 % of mooring-days (the profiler sites alone) and <code>sss</code> on 38.3 %, while <code>sst</code> is 78.9 % and <code>t_300m</code> 86.4 %. <code>qc</code> is 1 on 283,715 rows and 2 on 717,567
store.json	built 2026-09-14T07:01:04Z at commit <code>c5bc2ce</code> ; sha256 <code>50c6e57d7c887f6d2fecafde4c4354568070a99ab237e75073279c6bed81b7cb</code>
verdict	PASS on every check : 9 of 9 hashes, all 18 channels reproduced exactly, and the per-year counts reproduced exactly from <code>time_days</code>

And the profiler’s velocities are published in centimetres per second and stored in metres per second, with the factor read from the unit the service declares rather than hardcoded: the check is that the currents peak at 1.19 m s^{-1} and not at 119, because a silent factor of 100 on a current still looks like a plausible ocean.

5.3 socat — the Surface Ocean CO₂ Atlas

The longest record of the four, and the only one reaching back to 1957 — which is what the negative bins of §4 exist for.

`patm` is **never** `NCEP_SLP`. The file carries both the measured barometer and a reanalysis pressure interpolated to the ship — a model output wearing a measurement’s column — and substituting the second where the first is missing would be making up data, so `patm` is NaN there instead. The cost is visible: it is the only channel below 90 % measured, at 74.7 %, mean 1009.4 hPa over 859–1066 hPa. Two edge findings are flagged rather than failed: `fco2` sits exactly on its 2000 μatm bound on 6 rows, and `sss` is under 5 PSU on 128,336 rows of which 175 are exactly 0.0, which the builder’s $[0, 45]$ bound does not catch.

5.4 slatrack — along-track sea level

95.7 % of the family by row count, and the only store that needs credentials to fetch and a large machine to assemble. Section 6 takes it apart further.

6 slatrack in detail

Four things about this store are worth a section of their own: which satellites are in it, why one year is thin, where the archive has holes, and why its time column cannot be used as a clock.

6.1 Twenty-nine mission datasets, and the one with no rows

`platform` is `int(sha1(<mission dataset id>)[:15], 16)`. The verification hashed the 29 identifiers in `store.json`’s `plan.missions` and matched the store’s ids against them: **every row’s platform is**

source	the SOCAT v2026 synthesis release of 2026-06-16 (DOI 10.25921/8dba-fr90, NCEI accession 0315110): one 1,411,801,422-byte zip holding a 32-column tab-separated file behind an 8,454-line preamble, streamed and parsed in one pass
cadence	irregular — an underway measurement taken while the ship steams. Footprint ($-4.0, -4.0$)
channels ($C = 4$)	fco2 (μatm — fco2rec , SOCAT’s own recomputed and harmonised fugacity, not one of the six raw variants beside it); sst ($^{\circ}\text{C}$, the ship’s measured intake temperature); sss (PSU, measured practical salinity); patm (hPa, the measured barometric pressure PPPP)
quality control	two flags, both applied: the dataset-level QC.Flag keeps A–D and drops E, and the per-value World Ocean Circulation Experiment flag fco2rec.flag maps $2 \rightarrow 1, 3 \rightarrow 3, 4 \rightarrow 4, 9 \rightarrow 0$, with rows worse than 2 dropped. The published file states it holds only flag-2 values, so this is a guard rather than a filter — and the counters say how many rows each clause removed, because a guard that never fires and one that removes a third of the archive look identical from outside (both removed 0)
rows	$N = 41,830,675$ of 44,018,204 read (95.0%). drop_out_of_range 2,160,005 (outside the 1957–2024 window); drop_no_fco2 27,520
bins	$-1768 \dots 3141, 3,266$ of 4,910 live (66.5%) — the emptiest index of the four, the early record being a handful of cruises a decade; 1957-01-01 $\rightarrow$ 2024-12-31
years	68 listed, with true gaps (1958–60, 1964–67 and 1971–72 hold zero rows): 1957 = 578, past 1 M a year in 2005, 2024 = 2,359,858
platforms	8,024 distinct cruises, by hashed expocode — the unit is a <i>cruise</i> , not a ship, which is the right unit for holding out a calibration
measured	0.923146 overall; fco2 1.000000, sst 0.999875, sss 0.945353, patm 0.747356. qc is 1 on every row
store.json	built 2026-09-14T08:45:54Z, builder commit 0d5860d; sha256 aac9f3316ac7cb89ba4a3dabdb74dc22150043a72eeb11bd57c8895b782ad3f7
verdict	PASS on every check : 9 of 9 file hashes, all four channels reproduced exactly. One difference, and it is the format’s: recomputing each row’s calendar year from time.days differs from store.json ’s per_year by 1–3 rows in 16 of 68 years, because three rows late on 2024-12-31 round up across midnight in float32. store.json ’s counts, taken from the source dates, are the correct side

one of the **29**, and not one row of **2.03 billion** falls outside its own mission’s STAC coverage window, in either direction, at month granularity.

Missions contributing rise from **2** in 1993–1999 to **9** in 2023, and rows per day with them, 90k to 359k. That is the altimeter constellation’s own history, and why the per-year counts quadruple: a consumer training on this store is training on an observing system that grew.

6.2 January to March 1994 is one satellite

1994 is the only year that falls rather than rises: 29,146,039 rows against ≈ 33 M either side, a deficit of 3,835,024, and per-mission *monthly* counts locate it exactly. ERS-1’s 35-day repeat (**e1**) stops in mid-December 1993 — 1993-12 is itself a half month, 699,535 rows against a 1.31 M median — and does not resume until mid-March 1995, while its geodetic phase (**e1g**) starts 1994-04-10, exactly its STAC window’s first day. So the single-satellite interval is **1993-12-15 $\rightarrow$ 1994-04-09**, and January, February and March 1994 carry **TOPEX/Poseidon alone** — 1.59, 1.47 and 1.50 M rows against 2.94, 2.70 and 2.96 M a year earlier. With April’s partial start those four months are the whole deficit, and the record’s thinnest bin, 152,741 rows of 2,339, sits inside it. DUACS publishes no level-3 product for the 3-day ice-phase orbit ERS-1 flew in between, which is why the gap is a hole in **e1** rather than a thinner **e1**. **Nothing is missing from the build; the constellation was.**

6.3 GFO’s empty months

Scanning every mission for a calendar month lying wholly inside its own STAC window and carrying *zero* rows turns up two, both GFO: **2004-03** and **2006-09**, against a 1,435,239-row median month. Two more GFO months are far below that median without being empty (2003-09 at 257,212; 2007-02 at 161,740), and from 2007-02 the mission never again reaches 1.1 M in a month: the shape of a satellite degrading. It is flagged rather than passed over silently because this check cannot distinguish “the archive holds no file” from “the file was not asked for” from the outside; what it *can* say is that the fetch lane refuses to mark a year in which a listing came back empty or a download came back short, and all of 2000–2008 are marked. One other mission-month is comparable (**h2b** 2021-08, 272,078 against a 1,291,653 median). Treat GFO-era coverage as uneven.

source	Copernicus Marine Service product <code>SEALEVEL_GLO_PHY_L3_MY_008_062</code> , the DUACS reprocessed level-3 along-track sea level, as 29 per-mission datasets read as one table. The product’s public STAC (SpatioTemporal Asset Catalog) metadata gives every mission’s identifier, window and variable names with no credentials; the data needs them
cadence	1 Hz — one sample per second of flight, ≈ 7 km along the ground track. Footprint $(-2.0, -4.0)$
channels ($C = 3$)	<code>sla</code> (m — the archive’s <code>sla_filtered</code> , the low-pass-filtered anomaly), <code>sla_unfiltered</code> (m), <code>mdt</code> (m, the mean dynamic topography at the point, so absolute dynamic topography is the one addition <code>adt = sla + mdt</code>)
quality control	level 3 is edited upstream and publishes no per-sample flag, so every kept row carries <code>qc 1</code> and the store says so rather than inventing a grade. A sample whose <code>sla_filtered</code> is the netCDF fill or outside ± 3 m is dropped; the other two channels outside the same bounds become NaN alone
rows	$N = 2,030,800,150$ of $2,030,924,463$ read — 124,313 dropped , 0.0061 %, with <code>counts.out_of_bounds</code> empty, so no value was outside ± 3 m. The store names no rule for those 124,313; read against the builder, the only rule that can have produced them is the keep mask’s <code>isfinite(sla_filtered)</code>
bins	803 ... 3141, 2,339 of 2,339 live (100 %) ; 1993-01-01T00:00Z $\rightarrow$ 2024-12-31T23:50Z. Rows per live bin: 152,741 minimum, 745,348 median, 1,929,913 maximum
years	32, none empty: 1993 = 32,981,063 rising to 2023 = 130,879,712, with one fall — 1994 = 29,146,039 (§6)
platforms measured	28 distinct mission hashes carry rows, of 29 dataset identifiers in the plan 0.999052 overall: <code>sla</code> and <code>sla_unfiltered</code> on <i>every</i> row, <code>mdt</code> NaN on 5,774,351 (0.28 %), which is the whole of the store’s missingness
size	67.02 GB — twenty-two times the other three together, <code>platform.npy</code> (16,246,401,328 bytes) the largest file
store.json	built 2026-09-14T21:42:24Z, builder commit <code>6d67855</code> ; sha256 <code>d81b5e27dec71d47549850c442e58d3b88c16aae5e473a588c6c99de9220b12a</code>
verdict	PASS on every check , by <i>streaming</i> rather than download — 67 GB against 14 GB of sandbox disk, so all nine arrays were read over HTTPS in row-aligned lockstep and every check recomputed on the chunk in flight, one pass, 837 s at 80 MB/s. Being a pass over every row rather than a sample the checks are exhaustive: 0 descents in 2,030,800,149 adjacent <code>bin</code> pairs, 0 inside any bin on <code>time.days</code> , the index reproduced exactly, <code>bin == floor(time.days/5)</code> everywhere, and all three channels’ means reproduced to the seventeen digits <code>store.json</code> prints

6.4 The time column is coarser than the sampling, by $84\times$

`time.days` is float32. By 2015 the integer part needs 14 bits, leaving 10 for the fraction, so the column resolves 2^{-10} d = 84.4 seconds — and the instrument samples **once a second**.

bin	rows read	distinct <code>time.days</code>	rows per timestamp	float32 step at that t
803 (1993-01-01)	196,177	6,939	28.3	21.09 s
2411 (2015-01-03)	400,000	2,169	184.4	84.38 s
3141 (2024-12-31)	320,854	1,016	315.8	84.38 s

So 21 to 84 consecutive samples of one mission carry the identical timestamp, and at 6.5 km between samples that is 130 km of track in 1993 and **550 km in 2024**. Three consequences, all properties of the format rather than errors. The search’s Δt **cannot order two samples less than 84 seconds apart**: every token from one overflight arrives with the same age and the tie-break falls entirely to distance. The store is nonetheless correctly sorted — `time.days` is non-decreasing inside every one of the 2,339 bins, checked row by row — because the assembler’s stable sort preserved the parts’ own order inside each run of equal timestamps. And `bin` is exact, so **a calendar date must be derived from bin**. This is `socat`’s float32 finding one order of magnitude larger: `socat` loses 84 seconds against measurements minutes apart, `slatrack` against measurements one second apart. Nothing is hidden — the `schema` declares float32 — but this is the first store whose time column is coarser than its own sampling interval.

7 The build, and its guards

One builder, `ml/build_family10_stores.py`, with a source adapter per store and three stages: `index` lists the archive and writes the plan of files or years; `fetch` streams year by year and appends to per-year column parts; `publish` concatenates the parts in bin order, writes the index, computes hashes, uploads, and then **downloads each file back and re-hashes it** — a publish that cannot verify fails the job. Every one of the four build logs ends with `publish: 10 file(s) verified by restore`.

id suffix	mission	STAC window	rows	years present
tp	TOPEX/Poseidon	1992-10-13 → 2002-04-24	163,504,544	1993–2002
c2	CryoSat-2	2010-07-16 → 2020-07-31	161,609,793	2010–2020
alg	Saral/AltiKa geodetic orbit	2015-03-31 → 2026-01-16	150,903,772	2015–2024
s3a	Sentinel-3A	2016-06-28 → 2026-01-16	141,546,156	2016–2024
j2	Jason-2	2008-10-19 → 2016-05-26	136,145,157	2008–2016
en	Envisat	2002-05-17 → 2010-10-18	134,436,432	2002–2010
g2	GFO	2000-01-07 → 2008-09-07	128,066,623	2000–2008
j1	Jason-1	2002-04-24 → 2008-10-19	113,811,966	2002–2008
e2	ERS-2	1995-05-15 → 2002-05-14	109,297,139	1995–2002
s3b	Sentinel-3B	2018-11-27 → 2026-01-16	101,296,799	2018–2024
j3	Jason-3	2016-05-26 → 2021-12-29	99,342,274	2016–2021
c2n	CryoSat-2 new orbit	2020-08-01 → 2026-01-16	74,855,976	2020–2024
h2b	HaiYang-2B	2019-12-20 → 2026-01-16	70,829,087	2019–2024
h2ag	HaiYang-2A geodetic orbit	2016-03-31 → 2020-06-09	61,856,695	2016–2020
s6a-1r	Sentinel-6A low-rate mode	2021-12-29 → 2026-01-16	54,716,036	2021–2024
j1n	Jason-1 new orbit	2009-02-10 → 2012-03-03	53,323,537	2009–2012
j3n	Jason-3 interleaved orbit	2022-04-25 → 2025-01-07	48,608,308	2022–2024
tpn	TOPEX/Poseidon new orbit	2002-09-10 → 2005-10-03	46,627,866	2002–2005
al	Saral/AltiKa	2013-03-14 → 2015-03-31	34,309,111	2013–2015
h2a	HaiYang-2A	2014-04-12 → 2016-03-15	28,441,563	2014–2016
swon	SWOT nadir altimeter	2023-07-21 → 2026-01-16	22,545,701	2023–2024
enn	Envisat new orbit	2010-10-26 → 2012-04-08	21,472,828	2010–2012
j1g	Jason-1 geodetic orbit	2012-05-07 → 2013-06-01	19,699,076	2012–2013
e1	ERS-1	1992-10-23 → 1995-05-15	17,776,350	1993–1995
e1g	ERS-1 geodetic phase	1994-04-10 → 1995-03-21	14,825,066	1994–1995
j2n	Jason-2 interleaved orbit	2016-10-17 → 2017-05-17	10,093,265	2016–2017
swonc	SWOT nadir, calibration phase	2023-01-16 → 2023-07-09	7,475,129	2023
j2g	Jason-2 long-repeat orbit	2017-07-11 → 2017-09-14	3,383,901	2017
j3g	Jason-3 long-repeat orbit	2025-06-18 → 2026-01-16	0	—

Table 6: The 29 mission datasets. `j3g`’s zero is correct and not a gap: its orbit begins six months after the store’s last day, so a row from it would be the error.

Three stores on a free hosted runner. `gdp`, `gtmba` and `socat` are keyless and each fits a hosted runner’s six-hour cap and 14 GB of disk, so all three cost **\$0**: about 20 minutes each for `gtmba` and `socat`, 85 for `gdp` — and the 85 is the source, not the builder, because the drifter server serves one month per request and 46 years is about 550 sequential ones.

slatrack is built on two machines, because no one machine may be both. The Copernicus credentials may not live on a rented box, and a hosted runner cannot hold 67 GB. The seam is a prefix on the Hub itself. Six hosted lanes over weighted year ranges each fetch one year, push that year’s column parts to `partials/family10/slatrack/<year>/`, delete the local copy and move on — with `done.json` written **last**, and only after every file has been downloaded back with a matching hash, so a marker can only under-claim and a year killed halfway is simply refetched. A schedule every six hours is the resume mechanism. A keyless box then pulls every year back, verifies each file against its `done.json`, refuses by name if any year is missing, and assembles.

The streaming assembler. 2.03 billion rows do not fit in memory beside their own sort, so the store is assembled in three passes over the parts: count rows per bin to get the CSR offsets; scatter each part’s rows into `offsets[bin] + cursor[bin]` through a memory-mapped writer, in the order the parts are read; then sort each bin’s slice by `time_days` with a *stable* argsort. That is the same permutation the in-memory assembler’s `np.lexsort((time_days, bin))` produces, because `lexsort` is stable and its final tie-break is input order — which the scatter reproduces. A test hashes both assemblers’ output over a synthetic archive doctored to hold duplicate `(bin, time_days)` rows across two parts and asserts they are **byte-identical**. Peak memory is one part plus one bin’s slice, and a disk preflight computes the store’s size from the dtypes and refuses before pass 2 if free space is under $1.2\times$ it.

The assertions a store must pass before it is trusted. `check_store` runs on every assembled store and again before any publish: rows sorted by `(bin, time_days)`; `bin` equal to `floor(time_days / 5)`; the CSR offsets spanning exactly the rows, each bin’s slice holding only that bin; `lon` in $[-180, 180)$ and $|\text{lat}| \leq 90$; no infinity anywhere in `values`; the footprint columns constant; every channel inside its

stage	wall	what it did
index	36.6 s	the 29 mission windows, clipped to 1993–2024
parts pull	2,855 s	32 year-parts back off the Hub, each re-hashed; 0 missing
assemble	365.9 s	count rows per bin → CSR offsets (12.9 s), scatter 2,030,800,150 rows (240.9 s), stable sort of the 2,315 bins of 2,339 that needed one (112.1 s)
check + store.json	≈ 350 s	the chunked <code>check_store</code>
fetch total	3,579.4 s	peak resident memory 66.06 GB
publish	1,003.2 s	10 files, each verified by restore
registry	≈ 4 s	<code>family10.json</code> rewritten and verified by restore

Table 7: The `slatrack` assembly run: 1 h 18 m on a keyless 250 GB box at \$0.36/h, ≈ **\$0.55**, while the six credentialed lanes ran 10–16 minutes per year on hosted runners, so about 47 hours of downloading cost **\$0**. The estimate at dispatch was one box-day and ≈ \$8; splitting the credentialed half onto free parallel lanes is the difference.

declared bounds; and a nearest-neighbour search returning nothing from the future. Each is a property a broken build can have while looking entirely ordinary from outside, which is the only reason they are worth the seconds. Above 50 M part rows the check runs chunked, which is why it fits at all.

Three guards worth naming, because each closed a silent failure. `gtmba`’s build records `datasets_empty`: 37 — request-and-year combinations that came back with nothing — because a silent skip of seventeen mooring sites is invisible in a store that otherwise looks complete. The `slatrack` assembler refuses, *by name*, if any requested year is absent from the Hub prefix, so a partial store is only ever built deliberately, with `--allow-missing-years`. And a store built under any relaxation records it in a `degraded` block, so a consumer can tell a complete build from a permitted-partial one without reading a workflow log.

Two ledger bugs, found by arithmetic and fixed with the arrays untouched. `socat`’s first build attached the whole stream’s counters to every year part and the assembler summed them across the 59 years that held rows, so `rows_read` read 2,597,074,036 instead of 44,018,204 — exactly 59×. `gdp`’s `drogue_uncertain` ran 568 ahead of the NaN count in the channel it describes, and the row ledger failed to close by the same 568: one cause — 568 source fixes on which *nothing at all* was measured, counted and then discarded unrecorded — and two symptoms. Both stores were rebuilt the same day, and in both cases **all nine array sha256 values in the new store.json are byte-identical to the verified build’s**, so only the ledger changed and no measurement had to be re-checked. Both ledgers now close: `socat` reads 44,018,204 rows with 2,160,005 and 27,520 dropped, and `gdp` 48,708,540 – 227,174 – 568 – 48,480,798 = 0.

8 Reading a store

`ml/family10_store.py` is the only thing in the repository that opens a store, so the layout is defined once. It is pure numpy: no scipy, no torch.

```
from family10_store import Store

st = Store.open("tensors/family10/gdp")           # a directory
st = Store.open("chfrank/earth-tensors:tensors/family10/gdp") # or the Hub
tok = st.knearest(36.0, -70.0, bin=2411, k=8,
                  R_max_km=300.0, T_max_days=10.0)
```

`Store.open` takes a directory or a Hub prefix; the Hub form downloads `store.json` first and then exactly the files it names, never a whole-repo snapshot. `verify_store(dir)` re-hashes every file against `store.json`’s own record and raises on any mismatch — worth running, because a truncated `values.npy` still memory-maps and still answers a search, with whatever its tail happens to hold.

`knearest` returns a dict of **fixed length** k — always k slots, so a batch is never ragged: `values` (k, C) float32 in raw units with `mask` (k, C) beside it; `dx_km` and `dy_km`, signed kilometres east and north from the anchor, so the model never has to learn the cosine of latitude; `dt_days`, always ≥ 0 ; `dist_km`

and `d2`; `log2_fp` and `log2_dt` per returned row; `platform` and `qc`; `n_R`, the scalar count of observations inside the bounds — the local-density feature; `valid`, False marking a **miss token** (NaN values, `row = -1`); and `row`, `lat`, `lon`, `time_days`, `n_found`, `channels`.

Three rules are structural rather than applied afterwards. **One-sided in time**: only observations at or before the anchor’s pentad are candidates, because the CSR slice stops at the anchor’s own bin and never reads a later row — a forecaster that has seen the next pentad is not a forecaster. **Bounded** by `R_max_km` and `T_max_days`; where fewer than k exist inside them the remaining slots are miss tokens, the same token the gridded tensor uses at an empty cell, made rare instead of usual. And the **metric** is $d^2 = (dx^2 + dy^2)/L^2 + dt^2/T^2$ with L and T defaulting to the search bounds, which makes the two axes commensurate at the boundary; pass them explicitly to use a channel’s own correlation length. Δt is measured from the *end* of the anchor’s pentad, $5(\text{bin} + 1)$ days after the epoch, so an observation inside the anchor’s own bin is 0 to 5 days old rather than arriving from the future. `qc_max` narrows to the source’s best grade *before* `n_R` is counted: a token the reader will not return must not inflate the density feature.

A worked example. At 36°N , 70°W in bin 2411 with $k = 8$, $R_{\text{max}} = 300\text{ km}$, $T_{\text{max}} = 10\text{ d}$, `gdp` returns eight rows in 4.3 ms from **one** drogued drifter (identifier 116295) at exact 0.25-day steps, walking east over two days at $0.31\text{--}0.54\text{ m s}^{-1}$ with `sst` NaN throughout because that buoy’s thermistor is not reporting. `gtmba` returns eight miss tokens there in 0.2 ms, correctly — its array spans latitudes $[-25, 21]$ only — while at the TAO site 0°N , 140°W it returns the $0/+2/-2^\circ$ triplet at exactly $222.64 = 2 \times 111.32\text{ km}$ spacing and a monotone thermocline from 26°C to 11.6°C , where `gdp` returns eight miss tokens. **The two stores are complementary at that anchor, which is the point of the family.** Typical cost: `gtmba` 0.4 ms, `gdp` 4.3 ms, `socat` 11 ms.

Family 8’s Argo store opens through the same class, unchanged. It reads that store’s `temp` and `psal` blocks as one 32-column value matrix — temperature at the 16 Roemmich–Gilson pressures, then salinity at the same 16 — and takes the footprint constants out of its own `store.json`. The blocks stay separate memory maps and only the k selected rows are ever concatenated, so opening the 2.68 M-profile store costs nothing. The test suite asserts the result is **bit-identical** to the family-8 reader’s on every anchor it tries.

9 Limitations, and what is still open

The k tokens are usually one platform. Measured over 600 on-track anchors per store, the number of *distinct* platforms among the k returned observations has a median of **1 of 8** for `gdp`, **1 of 8** for `socat` and **2 of 4** for `gtmba`: the eight drifter slots are one buoy at six-hour intervals, the eight carbon slots one research cruise seconds apart along one line, and raising the radius does not change it. So k buys *one track’s redundancy* — genuine information about that track’s coherence, and not k nearby measurements of the ocean. Whether the search should be de-duplicated by `platform`, or k made much larger than 8, or neither, is **E-078b’s open question**: the answer depends on what the token is for, and nothing measured so far measures that. `slatrack` will be the extreme case — at 6.5 km between consecutive samples, any k under a few hundred is one overflight.

The suggested search radii were three to ten times too loose. At anchors drawn from each store’s own rows the k -th nearest observation sits far closer than the design assumed.

store	k	k -th neighbour, km (med./p90/p99)	age, d (med./p99)	measured (R, T)	old suggestion
<code>gdp</code>	8	19.2 / 68.4 / 104.3	2.0 / 3.8	$\approx 110\text{ km}, 4\text{ d}$	300 km, 10 d
<code>gtmba</code>	4	0.0 / 334.0 / 334.0	2.5 / 3.5	$\approx 350\text{ km}, 5\text{ d}$	1,500 km, 15 d
<code>socat</code>	8	4.5 / 46.2 / 111.3	1.8 / 4.9	$\approx 150\text{ km}, 30\text{ d}$	500 km, 30 d
<code>slatrack</code>	—	<i>not measured</i> — the measurement memory-maps the store, which is 67 GB			

Table 8: 2,000 anchors per store, generous search bounds (5,000 km, 60 d) with the ranking metric’s scales pinned to the old suggestion, seed 20260914.

`gtmba`’s 4th neighbour is at 0 km at the median because the array is a fixed lattice and the same mooring contributes several daily rows inside the window — the temporal bound, not the spatial one,

is what $k = 4$ actually spends. `socat`'s 30-day window is the one bound worth leaving generous: a ship revisiting a piece of ocean is rare, so a longer window is the only way a *second* cruise can ever enter the search.

Off-track, the miss token is the normal answer. At anchors drawn uniformly over the globe, the search returns **nothing at all** inside the suggested bounds on **71 %** of anchors for `gdp`, **81 %** for `gtmba` and **89 %** for `socat`, and widening R only makes the search slower. `mask` and `n_R` are *inputs*, not defects: $n_R = 0$ is a true statement about the observing system, and exactly what a gridded representation could not say. Do not impute, and do not fill NaN with zero — zero is a real fugacity and a real velocity.

Tier T is not built. The tile tier — a tile-date catalogue plus a small per-tile codec's cached tokens, held as a tier-P store — is designed and has no bytes. It is the one compute-bound step on the E-078 ladder, and it is argued on its own evidence rather than on this note's.

values is float16. About three decimal digits: $\approx 0.002^\circ\text{C}$ at 20°C and $\approx 0.5\mu\text{atm}$ at $700\mu\text{atm}$ — below every instrument's own uncertainty, and it halves the store. Cast to float32 before arithmetic; the reader already does.

Two provenance strings in `slatrack`'s `store.json` are stale. Its `verified` field still ends “Neither route can be run from this sandbox ... NOT YET MEASURED: the remote path layout of the original files”, and its `sources` names the `copernicusmarine` `subset` call. Both were written before any fetch had succeeded; the build that produced the store ran the `copernicusmarine.get-on-original-files` route end to end over 32 years. The product and the dataset identifiers are right, the call named is the one that was measured and rejected. Both are class constants, both are one-line fixes, and **no byte of any array is affected**.

The holdout protocol is unchanged. 2009, 2017 and 2023 are development years; train through 2020, test 2021–2024. The stores carry the *whole* record, pre-1982 included, and the loader restricts — which is why the negative bins are kept rather than dropped at build time.

10 Handover for an external agent

This section is written for an agent ingesting family 10 into its own infrastructure and needs no other document.

THE RULE: read only. The public dataset repository `chfrank/earth-tensors` on Hugging Face is your only source. You read it over anonymous HTTPS under `resolve/main/` — no account, no token; HTTP Range is supported, and the URL answers a 302 redirect to a content-delivery host, so follow redirects. **You must never write to that repository, and never to the `blauwelt/earth source repository`** — no upload, no commit, no pull request, no branch, no file of any kind, whatever the reason and however convenient it would be. Everything you need is readable. If something appears to require a write, it is out of scope: report it instead.

The single entry point.

```
https://huggingface.co/datasets/chfrank/earth-tensors/resolve/main/
tensors/family10/family10.json
```

Read it first. It lists all nine groups, and for each one the `tier`, the `prefix` under which its files live, and the files themselves with their `sha256`. Dispatch on `tier`: “G” means a bin-major array you read by byte range, “P” means a tier-P store you read with a k -nearest search. Everything below is reachable from that file; the tables here are a convenience and the registry is the authority. The copy described here is 45,610 bytes, `generated_utc` 2026-09-14T22:04:04Z, `sha256` 2e919d5abc66c91788f6dee991be64eba729690d50534f7a8715bc182d0dbdd6.

The tier-P files. Base URL, to which `<store>/<file>` is appended:

<https://huggingface.co/datasets/chfrank/earth-tensors/resolve/main/tensors/family10/>

Every store holds the same nine arrays plus `store.json`. Each array's sha256 is in that store's `store.json`; `store.json`'s own cannot be, so it is given here, measured on the bytes the Hub served on 2026-09-15.

file	gdp bytes	gtmba bytes	socat bytes	slatrack bytes
<code>bin.npy</code>	96,961,724	2,002,692	83,661,478	4,061,600,428
<code>time_days.npy</code>	193,923,320	4,005,256	167,322,828	8,123,200,728
<code>lat.npy</code>	193,923,320	4,005,256	167,322,828	8,123,200,728
<code>lon.npy</code>	193,923,320	4,005,256	167,322,828	8,123,200,728
<code>values.npy</code>	387,846,512	36,046,280	334,645,528	12,184,801,028
<code>platform.npy</code>	387,846,512	8,010,384	334,645,528	16,246,401,328
<code>qc.npy</code>	48,480,926	1,001,410	41,830,803	2,030,800,278
<code>fp.npy</code>	193,923,320	4,005,256	167,322,828	8,123,200,728
<code>bin_offsets.npy</code>	26,960	27,704	39,416	18,848
<code>store.json</code>	6,873	11,341	7,897	13,038
total	1.70 GB	63 MB	1.46 GB	67.02 GB

Table 9: 70.24 GB for the four stores. Every byte count is $128 + N \times \text{itemsizes} \times \text{columns}$, the 128 being the `.npy` header, so a size that does not match means a truncated file.

store	sha256 of <code>store.json</code>
<code>gdp</code>	<code>c3a14beb8d51db73f33636c9b805509e9d71721c9fe92d876b8c9621d4ab1e10</code>
<code>gtmba</code>	<code>50c6e57d7c887f6d2fecafde4c4354568070a99ab237e75073279c6bed81b7cb</code>
<code>socat</code>	<code>aac9f3316ac7cb89ba4a3dabdb74dc22150043a72eeb11bd57c8895b782ad3f7</code>
<code>slatrack</code>	<code>d81b5e27dec71d47549850c442e58d3b88c16aae5e473a588c6c99de9220b12a</code>

Verify before you trust. Four checks, in order of cost.

1. **sha256** of every array equals `store.json`'s own record, and `store.json`'s equals the table above.
2. **N from the file size.** Each array must satisfy $\text{bytes} = 128 + N \times \text{itemsizes} \times \text{cols}$ with N equal to `store.json`'s N , and `bin_offsets.npy` must hold `n.bins + 1` int64 entries. The `.npy` header declares its own dtype and shape; parse it and compare against `schema`.
3. **bin is monotonic** and never leaves `[bin_first, bin_last]`; `time_days` is non-decreasing inside each bin; and `bin == floor(time_days / 5)` on every row.
4. **The index is consistent:** `off[0]` is 0, `off[-1]` is N , the array is non-decreasing, and the cumulative sum of `bincount(bin - bin_first)` reproduces it exactly — equivalently, every row of the slice `off[i]:off[i+1]` has `bin == bin_first + i`.

All four were run on all four stores on 2026-09-14 and all four passed. If a store is too large to hold — `slatrack` is 67 GB — stream it: read the nine files over HTTPS in row-aligned lockstep, one connection per file, hashing each as its bytes arrive while one consumer recomputes the checks on the chunk in flight. That is how `slatrack` was verified, in one pass at 80 MB/s using 1.6 GB of memory; eight connections were dropped mid-stream by the content-delivery network and each was resumed by `Range` from the byte it had reached, which changes nothing, because the hash is over the byte sequence.

A minimal reader, in numpy alone, depending on nothing from the source repository:

```
import json, numpy as np

D = "tensors/family10/gdp" # a local copy of one store
meta = json.load(open(f"{D}/store.json"))
A = {n: np.load(f"{D}/{n}.npy", mmap_mode="r") for n in
      ("bin", "time_days", "lat", "lon", "values", "platform", "qc", "fp")}
```

```

off          = np.load(f"{D}/bin_offsets.npy")          # int64 [B+1], read whole
bin_first = meta["bin_first"]
channels    = [c["name"] for c in meta["channels"]]

def bin_of(date):
    # "2015-01-03" -> 2411
    d = (np.datetime64(date) - np.datetime64("1982-01-01")).astype("int64")
    return int(np.floor(d / 5.0))          # floor, so pre-1982 is negative

def rows_of_bin(b):
    # the CSR slice; may be empty
    i = b - bin_first
    return (slice(int(off[i]), int(off[i + 1]))
            if 0 <= i < len(off) - 1 else slice(0, 0))

def knearest(lat0, lon0, b, k=8, R_max_km=110.0, T_max_days=4.0):
    """The k nearest observations at or BEFORE bin b. One-sided by
    construction: the scan stops at bin b. dt runs from the END of bin b,
    so a row inside b is 0..5 days old, never negative."""
    KM, t_anchor = 111.32, 5.0 * (b + 1)
    b_lo = b - int(np.ceil(T_max_days / 5.0)) - 1    # bins the bound admits
    lo = rows_of_bin(max(b_lo, bin_first)).start
    hi = rows_of_bin(b).stop
    if hi <= lo:
        return None
    lat = np.asarray(A["lat"][lo:hi], np.float64)
    lon = np.asarray(A["lon"][lo:hi], np.float64)
    dt = t_anchor - np.asarray(A["time_days"][lo:hi], np.float64)
    dy = (lat - lat0) * KM
    dlon = np.mod(lon - lon0 + 180.0, 360.0) - 180.0 # the dateline is no wall
    dx = dlon * KM * np.maximum(np.cos(np.radians(0.5 * (lat + lat0))), 0.05)
    dist = np.hypot(dx, dy)
    ok = (dist <= R_max_km) & (dt >= 0.0) & (dt <= T_max_days)
    n_R = int(ok.sum())          # the local-density feature
    idx = np.flatnonzero(ok)
    if idx.size == 0:
        return {"n_R": 0, "valid": np.zeros(k, bool)} # k miss tokens
    d2 = ((dx[idx]**2 + dy[idx]**2) / R_max_km**2
          + dt[idx]**2 / T_max_days**2)
    idx = idx[np.argsort(d2, kind="stable")[:k]]
    rows = lo + idx
    v = np.asarray(A["values"][rows], np.float32)    # float16 -> float32
    fp = np.asarray(A["fp"][rows], np.float32)
    return {"values": v, "mask": np.isfinite(v), "channels": channels,
            "dx_km": dx[idx], "dy_km": dy[idx], "dt_days": dt[idx],
            "dist_km": dist[idx], "n_R": n_R,
            "log2_fp": fp[:, 0], "log2_dt": fp[:, 1],
            "platform": np.asarray(A["platform"][rows]),
            "qc": np.asarray(A["qc"][rows]),
            "valid": np.ones(len(rows), bool)}

tok = knearest(36.0, -70.0, bin_of("2015-01-03"))

```

Four things this code does that a naive version does not, each load-bearing. It floors on *floats*, so a date before 1982 lands in a negative bin rather than bin 0. It subtracts `bin_first` before indexing the index array. It wraps the longitude difference into $[-180, 180)$, so an anchor near the dateline still finds its neighbours. And it measures Δt from the end of the anchor's bin, so a row inside the anchor's own pentad reads as 0 to 5 days old rather than as arriving from the future. Pad to k slots with miss tokens if you want rectangular batches, and standardise from your training bins only.

How tier G points at family 7.2. The four tier-G groups are not stored under `tensors/family10/`. The registry’s `tier_g` block names the tensor by stem — `family7_global025_pentad_l2`, recipe `f712` — and gives its manifest URL; each group entry carries `prefix tensors/family7_global025_pentad_l2`, the file name, its byte count and sha256, plus `grid`, `shape`, `dtype`, `header_len`, `slab_bytes` and `bin_first`. A read is one HTTP range: for a bin-major array, `offset = header_len + (b - bin_first) × slab_bytes` and length `slab_bytes`. The response must be HTTP 206; a 200 means the host ignored the `Range` header and is about to send tens of gigabytes, and should be refused rather than consumed. The values are float16 and *z-scored*, so un-standardise them with the group’s `norm` out of the tensor’s `.npz`; the full account of that tensor is the family 7.2 design note, `ml/paper/notes/family72.pdf`. Verify tier-G files against `tensors/family7_global025_pentad_l2/manifest.json`, which is their authority; the registry’s copies were checked against it and matched 4 of 4.

Deprecation. `family7_global025_pentad_l1` will be **deleted from the Hugging Face dataset repository on 2026-10-15**. It is superseded in every respect by `f712`, whose three inherited files are the same bytes, and nothing in it is unique. Nothing in family 10 references it — `tier_g` names `f712` — so if your ingest points at `l1`, re-point it before that date. `family7_global025_pentad_l0` stays as the published original, because its hashes are cited elsewhere, but it carries a missing year of observed sea-surface temperature and sea ice and a non-reproducing atmosphere group, so it is a historical record rather than a recommendation. Pin `f712` by its hashes and re-read the manifest before a re-ingest.

Reporting a problem. Open an issue on the source repository, `github.com/blauwelt/earth` — a read-only agent can still open one, and an issue is not a write to the dataset. Quote the store, the file name, its sha256, the bin or row range, and the check that failed. If issues are unavailable to you, report to Chris.

References

- [1] Elipot, S. et al. (2016). A global surface drifter data set at hourly resolution. *J. Geophys. Res. Oceans* 121, 2937–2966. Data: NOAA AOML Global Drifter Program, 6-hourly quality-controlled interpolated product.
- [2] McPhaden, M. J. et al. (1998). The Tropical Ocean-Global Atmosphere observing system: a decade of progress. *J. Geophys. Res.* 103(C7), 14169–14240. Data: the TAO Project Office of NOAA/PMEL — TAO/TRITON, PIRATA and RAMA.
- [3] Bakker, D. C. E. et al. (2016). A multi-decade record of high-quality fCO₂ data in version 3 of the Surface Ocean CO₂ Atlas (SOCAT). *Earth Syst. Sci. Data* 8(2), 383–413. Release v2026, DOI 10.25921/8dba-fr90, NCEI accession 0315110. CC-BY 4.0; the SOCAT Fair Data Use Statement applies.
- [4] Copernicus Marine Service product `SEALEVEL_GLO_PHY_L3_MY_008_062` — DUACS reprocessed level-3 along-track sea level. E.U. Copernicus Marine Service Information.
bibitemf72 blauwelt/earth (2026). Family 7.2: the global tensor with ocean colour, a real elevation static, and a build that reproduces. Design note, 15 September 2026.
- [5] blauwelt/earth (2026). E-076: family 8 — the measurement is never missing, only far away. Plan and results, 7–8 September 2026.
- [6] blauwelt/earth (2026). E-078: family 10 — one schema for every granularity. Design note, 12 September 2026.